

Arquitectura de Datos

Especialización en Internet de las Cosas

Docente:

Lacheski, Martín Anibal

Clase 5:

Bases de Datos NoSQL

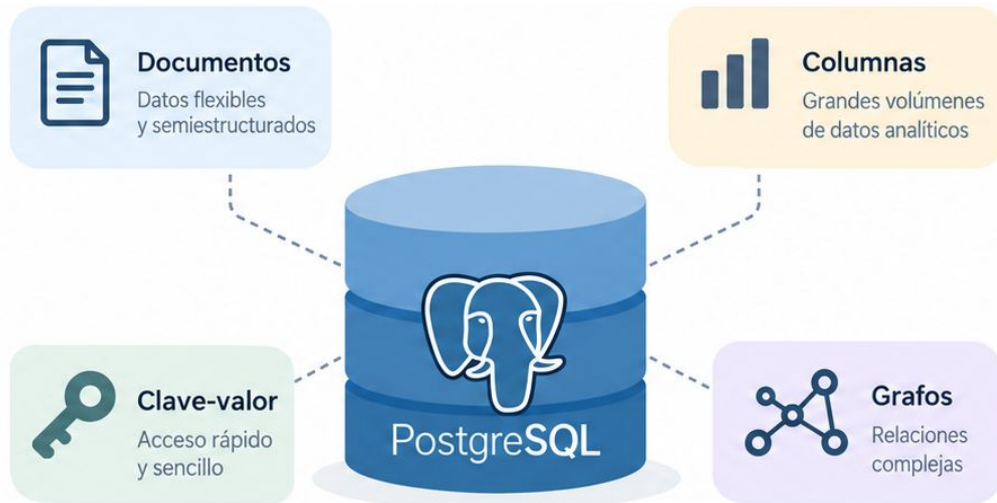


Arquitectura de Datos

Ya sabemos trabajar con una base relacional

Hasta ahora aprendimos a:

- modelar información;
- implementar estructuras relacionales;
- consultar mediante SQL;
- normalizar;
- crear índices;
- analizar planes de ejecución;
- utilizar JSONB;
- evolucionar el esquema.



Ahora aparece una nueva pregunta:

¿qué ocurre cuando el modelo relacional no es necesariamente la única alternativa?



**NoSQL no reemplaza lo aprendido.
Amplía las alternativas disponibles.**

Mejoramos la base relacional

Hoy vamos a comparar modelos diferentes

Hasta Clase 4:



Clase 5:



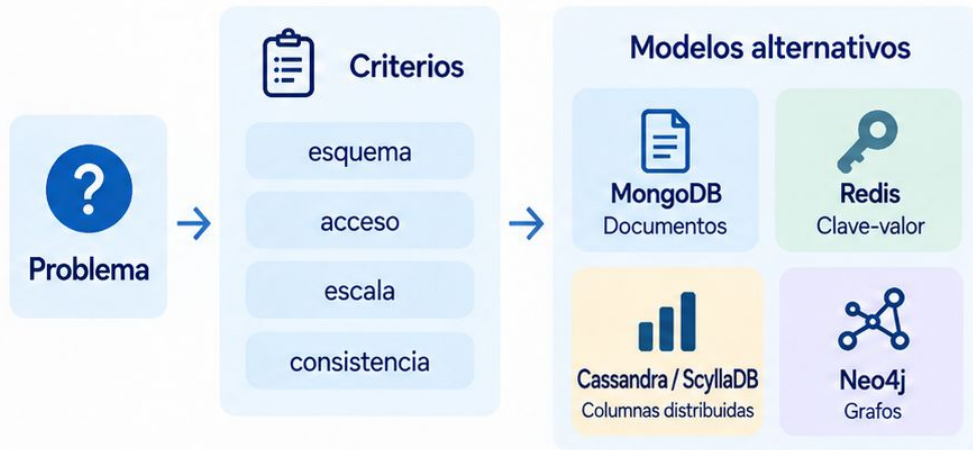
Antes de optimizar una tecnología debemos preguntarnos si su modelo representa bien el problema.

Objetivos de la clase

Comprender cuándo otros modelos pueden resultar útiles

Vamos a aprender a:

- comprender por qué aparecieron las bases NoSQL;
- analizar flexibilidad de esquema;
- relacionar distribución, disponibilidad y consistencia;
- comprender conceptualmente CAP;
- modelar mediante documentos;
- comprender el modelo clave-valor;
- analizar bases columnares distribuidas;
- representar información mediante grafos;
- comparar SQL y NoSQL;
- combinar tecnologías cuando exista una necesidad real.



El objetivo no es elegir “la mejor base”.

Es reconocer qué modelo responde mejor a determinados requerimientos.

El caso también evoluciona

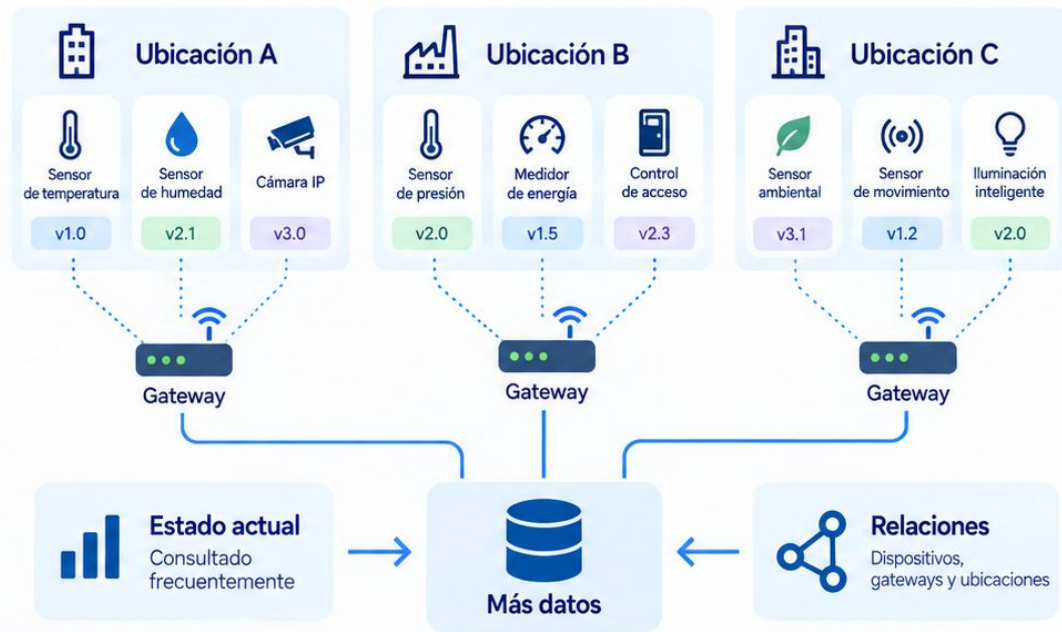
El mismo caso comienza a presentar nuevas necesidades

El sistema de monitoreo ahora posee:

- muchos modelos de dispositivos;
- configuraciones diferentes;
- dispositivos distribuidos;
- estado actual consultado frecuentemente;
- relaciones entre dispositivos, gateways y ubicaciones;
- mayor volumen de información.

La base relacional sigue funcionando.

Pero algunas necesidades pueden resolverse de otras maneras.



La aparición de NoSQL debe responder a un problema concreto, no a una moda tecnológica.

Arquitectura de Datos

Agenda del día

- ¿Por qué aparecieron las bases NoSQL?
- Flexibilidad de esquema.
- Distribución, disponibilidad y consistencia.
- Introducción a CAP.
- Bases documentales.
- Bases clave-valor.
- Bases columnares distribuidas.
- Bases de grafos.
- JSONB vs. base documental.
- SQL vs. NoSQL.
- Persistencia polígota.
- Práctica de selección y modelado.



Vamos a estudiar modelos diferentes observando qué decisiones de arquitectura cambia cada uno.

1

¿Por qué NoSQL?



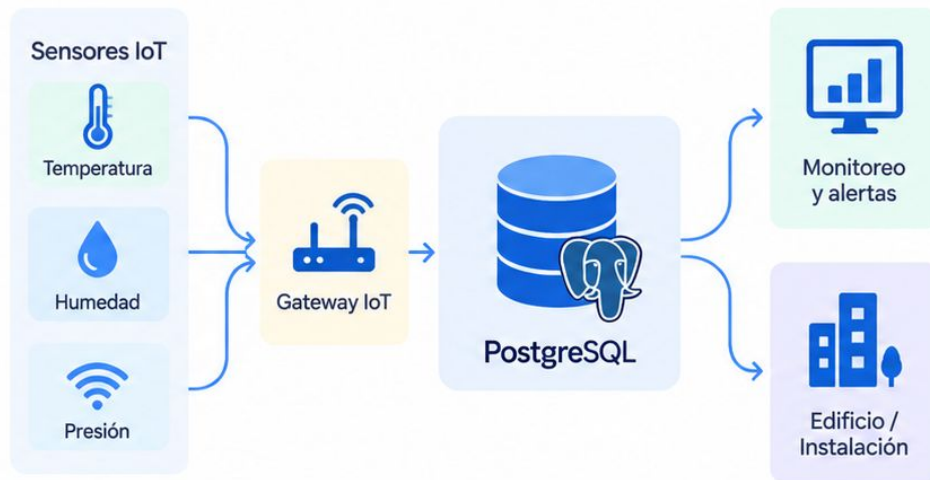
NoSQL no reemplaza a SQL

El modelo relacional continúa resolviendo muchísimos problemas

Una base relacional resulta especialmente útil cuando necesitamos:

- estructura bien definida;
- relaciones claras;
- integridad;
- transacciones;
- consultas flexibles;
- consistencia entre diferentes entidades.

Nuestro caso guía puede implementarse perfectamente en PostgreSQL.



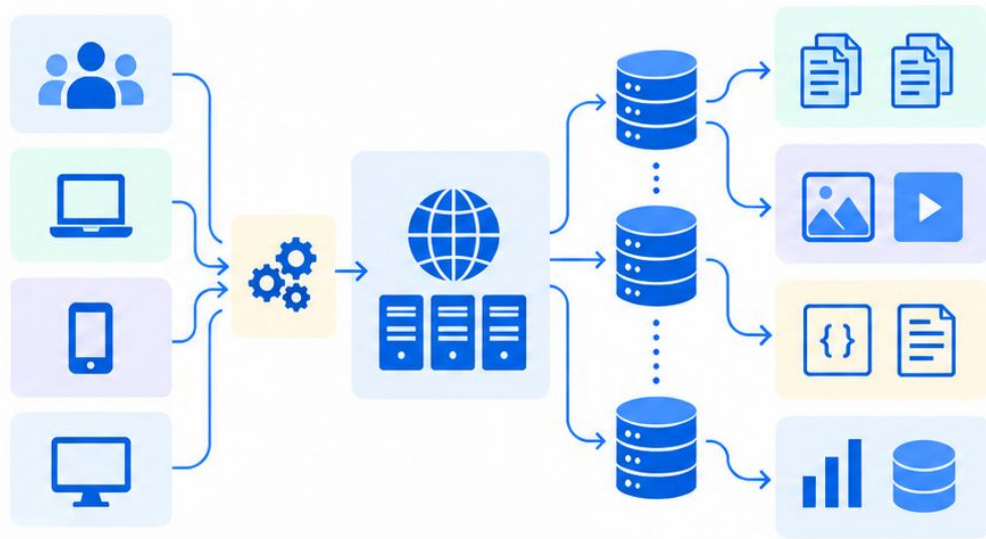
La existencia de nuevas alternativas no invalida las anteriores.

Nuevos sistemas, nuevas tensiones

Algunos requerimientos comenzaron a dominar el diseño

Con el crecimiento de sistemas web y distribuidos aparecieron escenarios con:

- enormes volúmenes;
- grandes cantidades de operaciones;
- datos heterogéneos;
- estructuras que cambian frecuentemente;
- distribución entre múltiples nodos;
- requerimientos elevados de disponibilidad.



NoSQL surge como respuesta a distintos problemas.
No existe una única razón ni un único modelo NoSQL.

Cuando aparecen fricciones

Una estructura puede resultar incómoda, no necesariamente incorrecta

Supongamos que cada modelo de dispositivo posee parámetros completamente diferentes.

Podemos resolverlo relacionamente.
Pero quizás aparecen:

- muchas columnas opcionales;
- estructuras auxiliares;
- cambios frecuentes del esquema;
- información profundamente anidada.

Otra representación podría resultar más natural.



Tabla relacional

id	modelo	wifi_ssid	bateria_ma	resolucion	gps	puertos
1	A1	Oficina	5000	NULL	NULL	USB-C
2	B2	NULL	NULL	1080p	Sí	NULL
3	C3	NULL	3000	NULL	Sí	USB-C
4	D4	Casa	NULL	4K	NULL	HDMI
5	E5	NULL	NULL	NULL	Sí	NULL



Documento

```
{
  "modelo": "D4",
  "resolucion": "4K",
  "puertos": [
    "HDMI",
    "USB-C"
  ],
  "camara": {
    "frontal": true,
    "trasera": true
  },
  "audio": {
    "microfono": true
  }
}
```



Que algo pueda implementarse no significa que sea la representación más conveniente.

NoSQL

Una familia de modelos diferentes

Documental

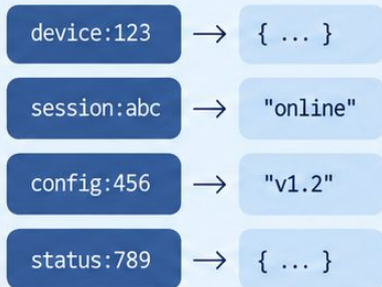
→ documentos y colecciones



Ejemplo: MongoDB

Clave-valor

→ clave → valor



Ejemplo: Redis

Columnas distribuidas

→ particiones organizadas para patrones de acceso

clave de partición	columna1	columna2	...
dispositivo:001	...	...	...
dispositivo:002	...	...	...
dispositivo:003	...	...	...
...	...	...	...

Ejemplo: Cassandra / ScyllaDB

Grafos

→ nodos y relaciones



Ejemplo: Neo4j



“NoSQL” agrupa tecnologías que pueden tener modelos y objetivos muy diferentes.

Diferentes decisiones

Cada modelo facilita ciertos tipos de problemas

Documentos

Estructuras variables o jerárquicas.



¿Cómo representar información variable?

Clave-valor

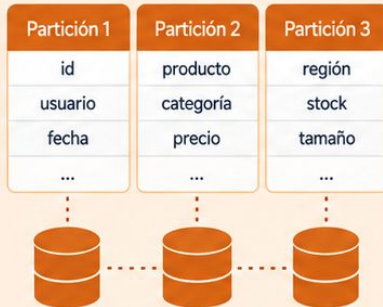
Acceso directo mediante una clave.



¿Cómo recuperar un valor rápidamente?

Columnas distribuidas

Escala y patrones de acceso conocidos.



¿Cómo escalar consultas previsibles?

Grafos

Relaciones y recorridos complejos.



¿Cómo recorrer relaciones complejas?



Cambiar el modelo de almacenamiento también cambia la forma en que pensamos los datos.

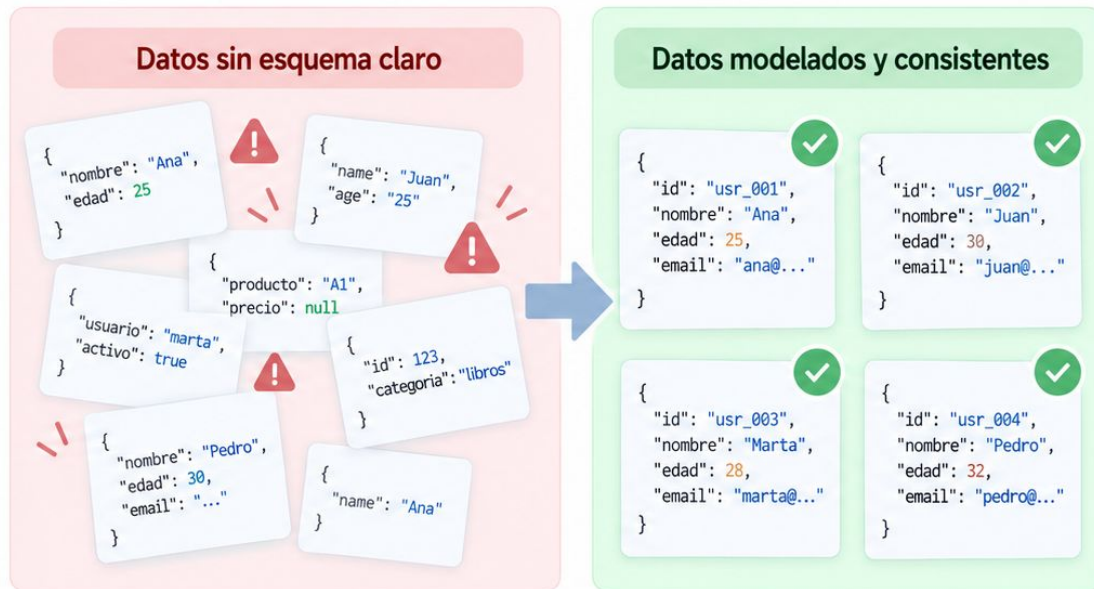
Un error frecuente

“No tiene esquema, guardamos cualquier cosa”

Una estructura flexible todavía necesita decisiones sobre:

- nombres;
- tipos;
- identificadores;
- campos obligatorios;
- versiones;
- relaciones;
- validación;
- patrones de consulta.

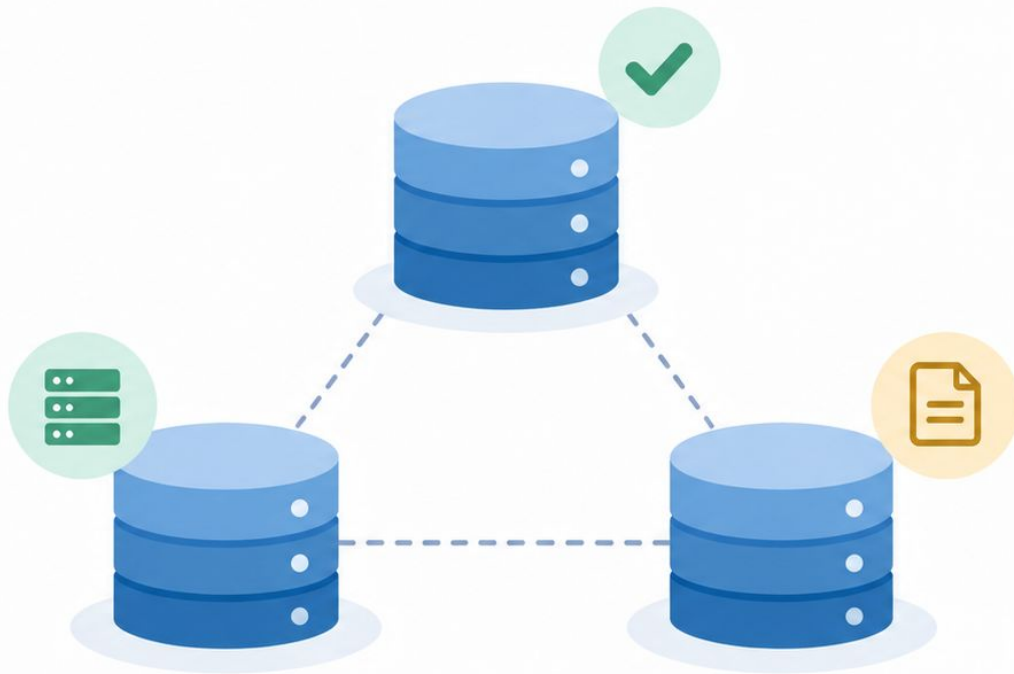
La diferencia es **dónde y cómo** se aplican esas reglas.



NoSQL reduce algunas rigideces.
No elimina la necesidad de modelar.

2

Distribución, Disponibilidad y Consistencia



Escalabilidad horizontal

Incorporar nodos en lugar de depender de una única máquina

Escalabilidad vertical

Potenciar una única máquina



Escalabilidad horizontal

Incorporar más nodos



Distribuir datos permite crecer de otra manera, pero también introduce nuevos problemas.

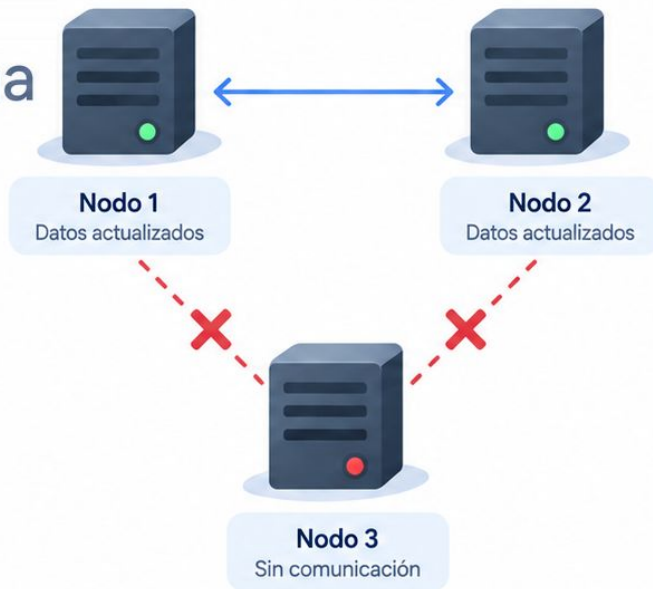
Distribuir cambia el problema

Ahora existen varias copias y varios nodos

La red pasa a formar parte del sistema

En un sistema distribuido pueden ocurrir situaciones como:

- un nodo deja de responder;
- la comunicación se vuelve lenta;
- dos nodos dejan temporalmente de comunicarse;
- diferentes nodos conocen estados distintos;
- una operación alcanza solamente parte del sistema.



Cuando distribuimos datos, los fallos de comunicación se convierten en parte normal del diseño.

Disponibilidad

¿Podemos seguir respondiendo?

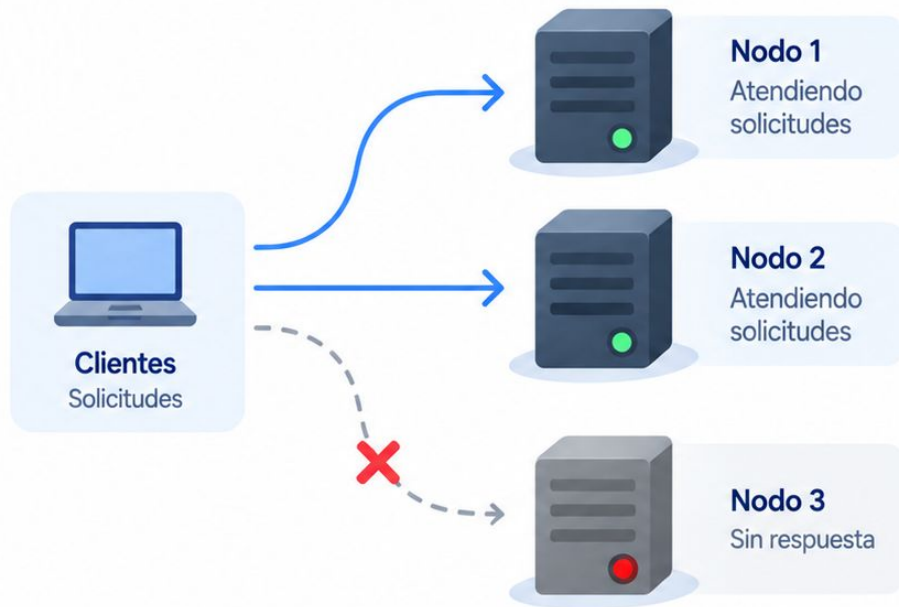
Una aplicación disponible intenta continuar atendiendo operaciones incluso cuando aparecen fallos parciales.

Ejemplo:

un nodo del sistema deja de responder.

La pregunta es:

¿otros nodos pueden continuar atendiendo solicitudes?



Disponibilidad significa que el sistema pueda seguir ofreciendo una respuesta ante las solicitudes.

Consistencia

¿Qué versión del dato observa cada cliente?

Estado inicial:

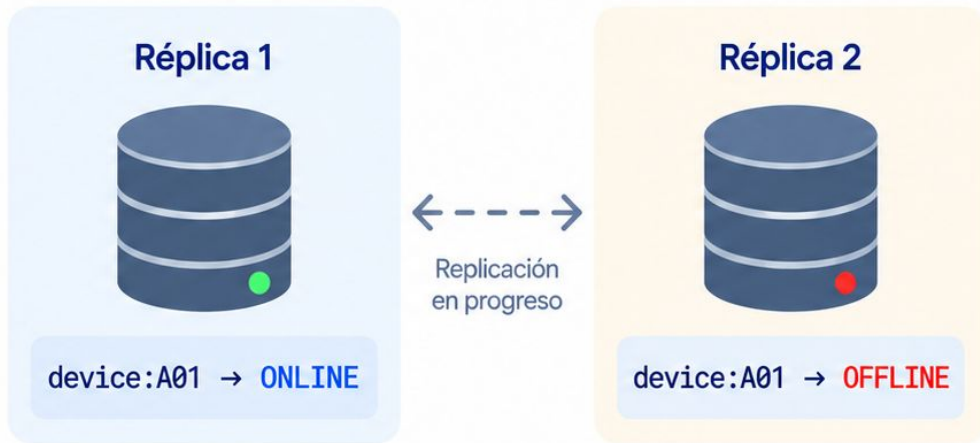
device:A01 → ONLINE

Una aplicación modifica:

device:A01 → OFFLINE

En un sistema distribuido debemos preguntarnos:

¿todos los nodos observan inmediatamente el nuevo valor?



Distribuir datos obliga a definir qué garantías necesitamos respecto de las distintas copias.

La red puede dividirse

Los nodos continúan existiendo, pero no pueden comunicarse

En un sistema distribuido los nodos pueden perder la comunicación entre sí, aunque sigan funcionando.

A esta situación la podemos analizar como una **partición de red**.

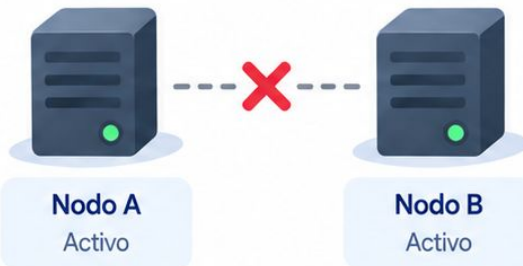
Funcionamiento normal

Los nodos se comunican



Partición de red

La comunicación falla



Ambos nodos siguen activos, pero no pueden comunicarse entre sí.



Un sistema distribuido debe decidir cómo comportarse cuando algunas partes dejan de comunicarse.

Teorema CAP

Una idea fundamental para sistemas distribuidos

Ante una partición de red, un sistema distribuido debe enfrentar una tensión entre:

C

Consistencia

Responder utilizando una visión coherente del dato.

A

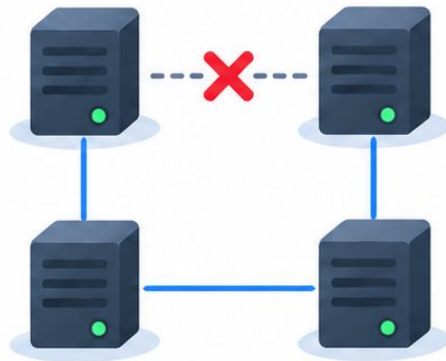
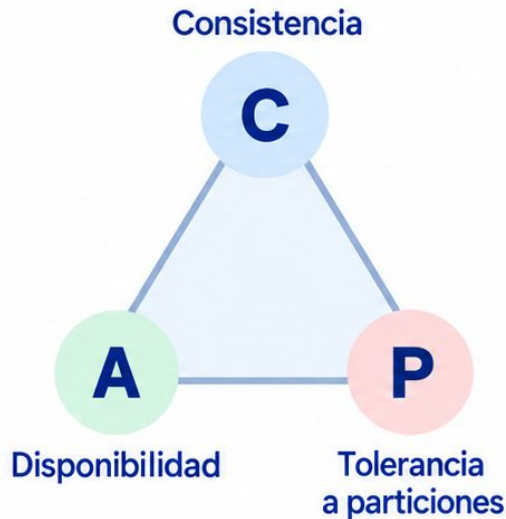
Disponibilidad

Continuar respondiendo solicitudes.

P

Tolerancia a particiones

Poder operar en un entorno donde la comunicación entre nodos puede fallar.



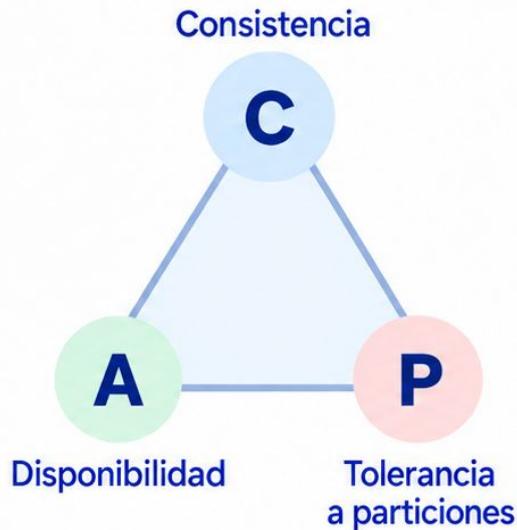
CAP analiza qué garantías pueden mantenerse cuando ocurre una partición de red.

Una simplificación frecuente

No se trata de marcar dos casillas para siempre

En sistemas distribuidos reales:

- las particiones pueden ocurrir;
- diferentes operaciones pueden requerir garantías distintas;
- las tecnologías permiten configurar diferentes comportamientos;
- pueden existir distintos niveles de consistencia.



La pregunta útil no es “¿qué dos letras elijo?”. Es “¿qué debe ocurrir cuando la red falla?”.

Un ejemplo IoT

Dos nodos dejan de comunicarse

Supongamos que el estado de un dispositivo cambia a `OFFLINE`.

Durante una partición:

- **Alternativa A**
Rechazar determinadas operaciones hasta poder asegurar el estado correcto.
- **Alternativa B**
Continuar respondiendo aunque temporalmente diferentes nodos puedan conocer valores distintos.

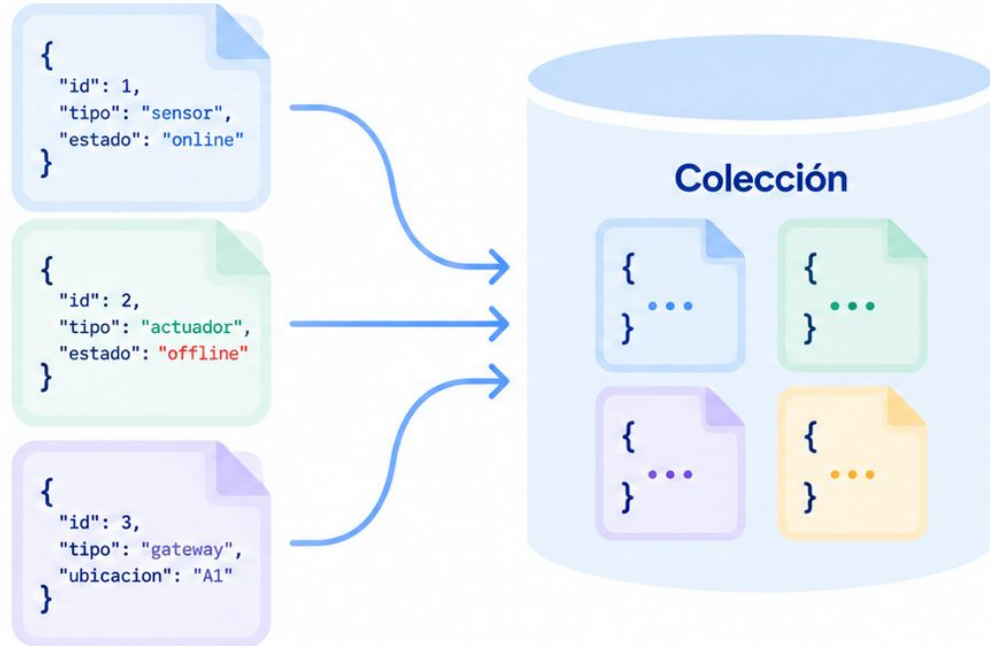
La elección depende del requerimiento.



Consistencia y disponibilidad son requisitos del problema, no características que debemos maximizar siempre.

3

Bases Documentales



Modelo documental

Representar información relacionada dentro de una estructura

```
{  
  "device_id": "AMB-001",  
  "modelo": "EnvMonitor-X",  
  "ubicacion": "Aula 204",  
  "configuracion": {  
    "intervalo": 60,  
    "variables": ["temperatura", "humedad"]  
  }  
}
```

device_id

Identificador del dispositivo

modelo

Modelo del dispositivo

ubicacion

Lugar donde se encuentra

configuracion

Información relacionada
dentro de un mismo bloque



Un documento puede representar información estructurada y anidada dentro de una misma unidad.

Documentos y colecciones

Otra manera de organizar información

Colección:

dispositivos

contiene:

- Documento A
- Documento B
- Documento C
- ...

Los documentos de una colección representan elementos relacionados conceptualmente.



Una colección agrupa documentos, pero esos documentos no necesitan ser idénticos campo por campo.

Estructuras variables

Una aplicación frecuente en IoT

colección dispositivos

Dispositivo A:

```
{  
  "intervalo": 60,  
  "temperatura": true  
}
```

Dispositivo B:

```
{  
  "intervalo": 30,  
  "temperatura": true,  
  "humedad": true,  
  "presion": true,  
  "calibracion": {  
    "offset": 0.4  
  }  
}
```

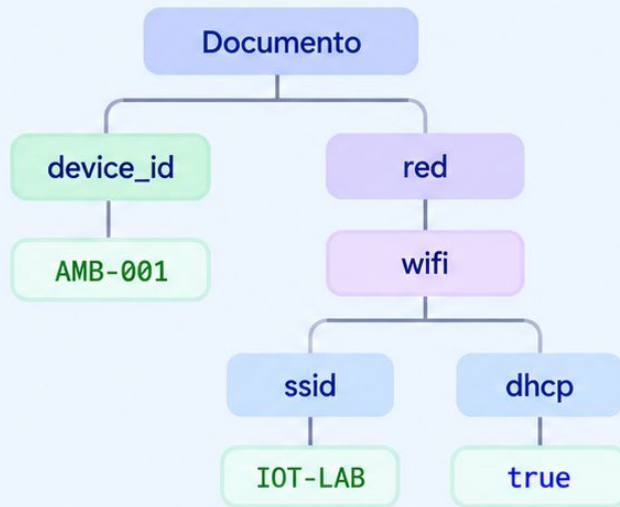


La estructura documental permite que distintos elementos posean atributos diferentes.

Documentos jerárquicos

La estructura puede conservar relaciones internas

```
{  
  "device_id": "AMB-001",  
  "red": {  
    "wifi": {  
      "ssid": "IOT-LAB",  
      "dhcp": true  
    }  
  }  
}
```



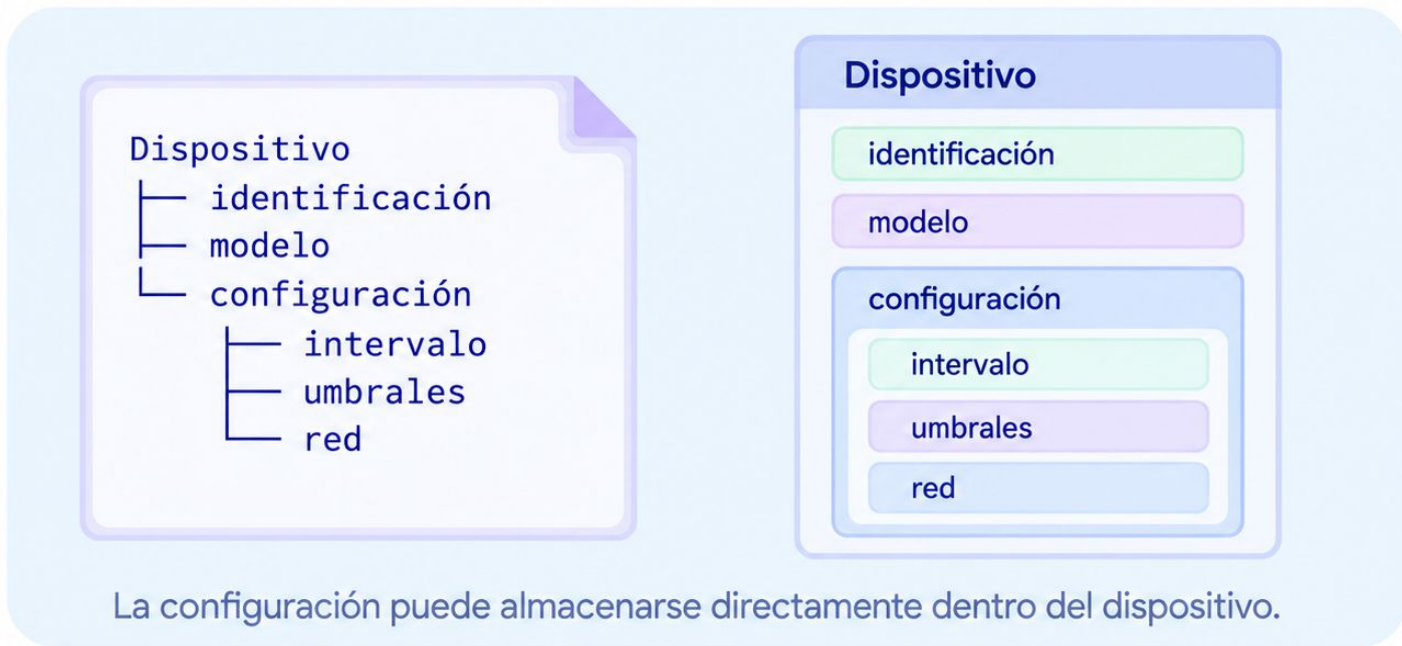
La información relacionada puede mantenerse agrupada dentro del propio documento.



No toda estructura jerárquica necesita convertirse inmediatamente en varias tablas.

Persistencia en el documento

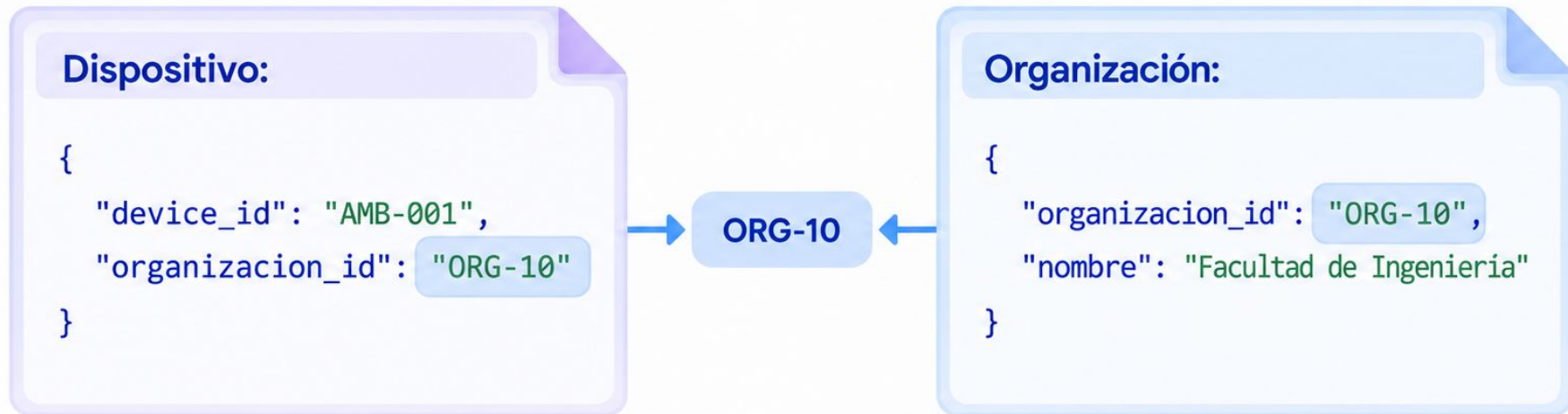
Cuando los datos pertenecen naturalmente al mismo agregado



Embebemos información cuando normalmente se consulta y modifica junto con el documento principal.

No todo debe estar embebido

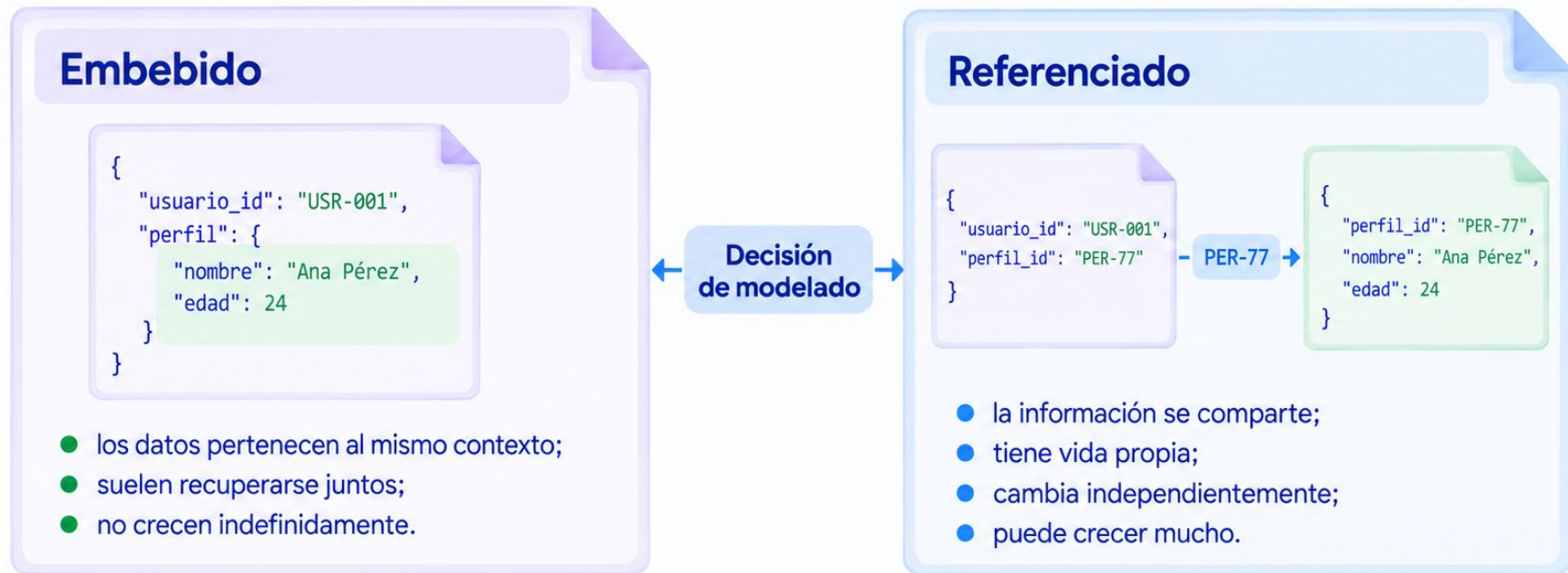
Los documentos también pueden relacionarse



Un modelo documental también debe decidir qué información incorporar y qué información referenciar.

Una decisión de modelado

Pensar cómo se utiliza la información



La estructura del documento debe acompañar los patrones de acceso.

¿Qué necesitamos recuperar?

El acceso influye en la estructura

Pregunta frecuente:

“Obtener un dispositivo junto con toda su configuración.”

Un documento que contenga:

Dispositivo + Configuración

puede resolver la consulta de manera directa.

Otra necesidad podría requerir una estructura distinta.



Resultado

Documento

Dispositivo con su configuración

```
{
  "device_id": "AMB-001",
  "modelo": "EnvMonitor-X",
  "ubicacion": "Aula 204",
  "configuracion": {
    "intervalo": 60,
    "variables": ["temperatura", "humedad"],
    "umbrales": {
      "temperatura": { "min": 0, "max": 40 },
      "humedad": { "min": 20, "max": 80 }
    }
  }
}
```



En NoSQL, los patrones de consulta suelen influir fuertemente en el modelado.

La estructura puede evolucionar

No todos los documentos necesitan cambiar al mismo tiempo

Supongamos que aparece un nuevo dispositivo con:

`firmware_auto_update`

Los documentos nuevos pueden incorporar ese campo sin que todos los anteriores necesiten tenerlo inmediatamente.

La aplicación debe saber interpretar ambas versiones.

Documento v1
Dispositivo existente

```
{
  "device_id": "AMB-001",
  "modelo": "EnvMonitor-X",
  "ubicacion": "Aula 204",
  "configuracion": {
    "intervalo": 60,
    "umbral": 75
  }
}
```



Documento v2
Nuevo dispositivo

```
{
  "device_id": "AMB-002",
  "modelo": "EnvMonitor-Y",
  "ubicacion": "Laboratorio",
  "configuracion": {
    "intervalo": 60,
    "umbral": 75
  },
  "firmware_auto_update": true
}
```

Ambas versiones
pueden coexistir



Flexibilidad de esquema facilita ciertos cambios, pero traslada parte de la responsabilidad a la aplicación y al diseño.

Flexible no significa inconsistente

La aplicación sigue necesitando reglas

Podemos requerir que:

- `device_id` exista;
- `intervalo` sea positivo;
- determinadas configuraciones tengan tipos correctos;
- ciertas combinaciones de campos sean válidas;
- la versión del documento sea conocida.



Permitir estructuras diferentes no significa aceptar estructuras incorrectas.

Parecidos, pero no equivalentes

Guardar documentos no convierte a PostgreSQL en una base documental

PostgreSQL + JSONB

- JSONB es un tipo de dato dentro de PostgreSQL;
- convive con tablas, columnas, relaciones y SQL;
- podemos combinar estructura relacional y documentos.

tabla

id	nombre	email	datos (JSONB)
1	Ana	ana@ejemplo.com	<pre>{ "rol": "admin", "preferencias": { "tema": "oscuro", "idioma": "es" } }</pre>
2	Luis	luis@ejemplo.com	<pre>{ "rol": "usuario", "notificaciones": true, "ciudad": "Madrid" }</pre>
3	Marta	marta@ejemplo.com	<pre>{ "rol": "editor", "equipos": ["datos", "web"] }</pre>

Base documental

- el documento constituye el modelo principal;
- los documentos se organizan en colecciones;
- el motor está diseñado alrededor de ese paradigma.

colección

```
{
  "_id": "a1",
  "nombre": "Ana",
  "rol": "admin",
  "preferencias": {
    "tema": "oscuro"
  }
}
```

```
{
  "_id": "b2",
  "nombre": "Luis",
  "rol": "usuario",
  "notificaciones": true,
  "ciudad": "Madrid"
}
```

```
{
  "_id": "c3",
  "nombre": "Marta",
  "rol": "editor",
  "equipos": [
    "datos",
    "web"
  ]
}
```



JSONB permite incorporar documentos a un modelo relacional.
Una base documental organiza el sistema principalmente alrededor de documentos.

¿Cuándo puede resultar útil?

Cuando la estructura acompaña al agregado

Por ejemplo:

- configuraciones variables;
- catálogos heterogéneos;
- metadatos;
- información jerárquica;
- perfiles con atributos diferentes;
- datos que normalmente se recuperan juntos.



Sensor de temperatura

```
{
  "device_id": "AMB-001",
  "tipo": "temperatura",
  "intervalo": 60,
  "unidad": "°C",
  "umbral_max": 35
}
```



Sensor de humedad

```
{
  "device_id": "AMB-002",
  "tipo": "humedad",
  "intervalo": 30,
  "unidad": "%",
  "calibracion": {
    "offset": 0.4
  }
}
```



Estación meteorológica

```
{
  "device_id": "AMB-003",
  "tipo": "estacion_meteorologica",
  "intervalo": 60,
  "variables": [
    "temperatura",
    "humedad",
    "presion",
    "viento"
  ],
  "ubicacion": {
    "lat": -27.47,
    "lon": -55.88
  }
}
```



El modelo documental puede ser útil cuando un documento representa naturalmente una unidad de información.

¿Cuándo tener cuidado?

La flexibilidad tiene costos

Debemos analizar especialmente:

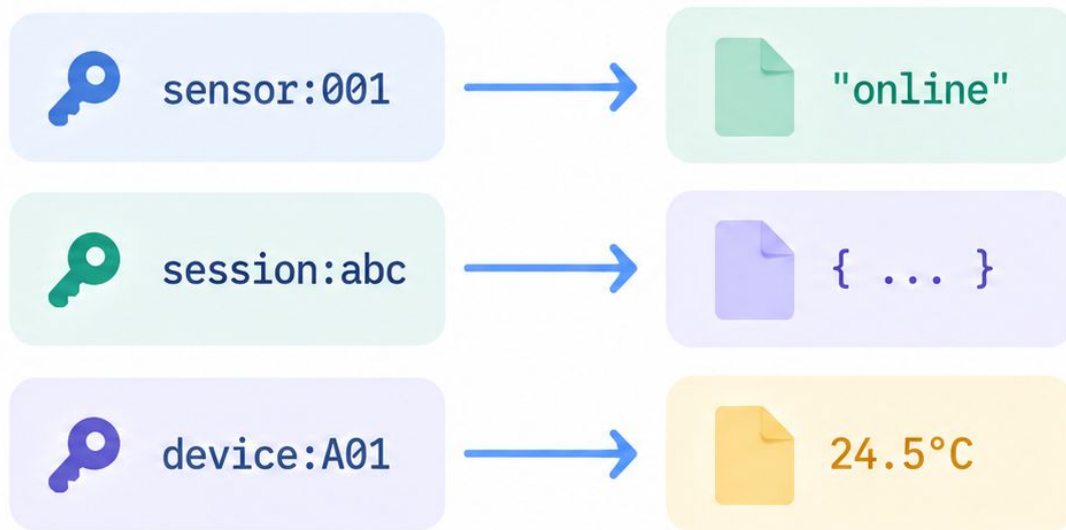
- duplicación de información;
- actualizaciones en muchos documentos;
- relaciones complejas;
- documentos que crecen demasiado;
- consultas que atraviesan múltiples entidades;
- consistencia entre datos duplicados.



Un documento demasiado grande o demasiado conectado puede indicar que el modelo necesita revisarse.

4

Bases Clave-Valor



Diseñar las claves

Debe representar cómo accederemos al dato

Ejemplos de claves:

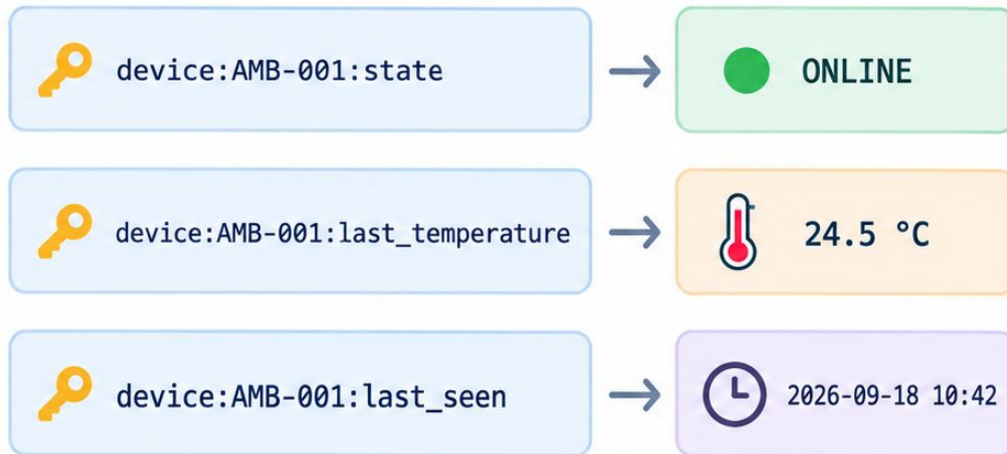
device:AMB-001:state

device:AMB-001:last_temperature

device:AMB-001:last_seen

Una convención clara ayuda a:

- identificar;
- organizar;
- recuperar;
- eliminar;
- expirar datos.



En un modelo clave-valor, diseñar las claves es parte del modelado.

Estado actual

Guardar el último estado conocido

Mediciones históricas:

●	10:30	→	24,2 °C
●	10:31	→	24,4 °C
●	10:32	→	24,6 °C



Estado actual:



device:AMB-001:last_temperature



24.6



Histórico y último valor responden preguntas diferentes y pueden utilizar mecanismos diferentes.

Clave → Valor

Recuperar información mediante un identificador conocido



La operación principal es conceptualmente:
conozco la clave → obtengo el valor



El modelo clave-valor simplifica el acceso cuando conocemos exactamente qué dato queremos recuperar.

TTL

Información que solamente tiene valor durante cierto tiempo

Podemos necesitar que un dato desaparezca automáticamente luego de:

- segundos;
- minutos;
- horas;
- determinado período.

Ejemplos:

- sesiones;
- resultados temporales;
- caché;
- estado de presencia;
- tokens.

Ejemplo:



Algunos datos tienen una vida útil definida desde el momento en que se almacenan.

Caché

Guardar temporalmente información de acceso frecuente

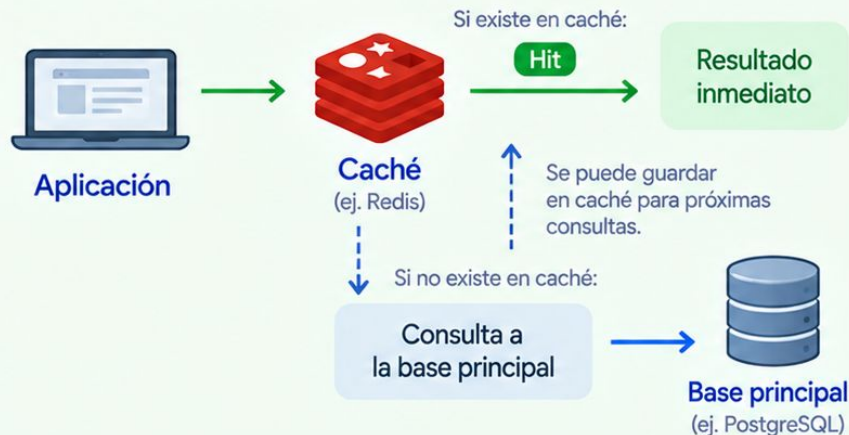
Sin caché

Cada consulta debe ir a la base principal.



Con caché

Primero se consulta la caché.



Una caché busca reducir latencia o trabajo repetido, no necesariamente reemplazar la persistencia principal.

¿Qué ocurre si desaparece?

Distinguir datos temporales de datos persistentes

Si **podemos reconstruir el dato desde otra fuente**, quizás estamos ante una caché.

Ejemplo:



PostgreSQL



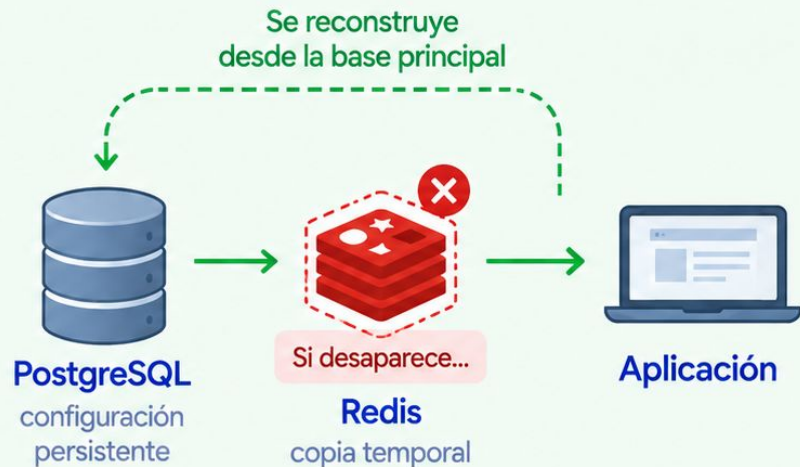
configuración
persistente



Redis



copia temporal
utilizada frecuentemente



Antes de utilizar almacenamiento temporal debemos saber si el dato puede perderse o reconstruirse.

Una operación sencilla

Guardar y recuperar estado

Consola Redis

```
127.0.0.1:6379> SET device:AMB-001:state ONLINE
OK

127.0.0.1:6379> GET device:AMB-001:state
"ONLINE"

# Con expiración (60 segundos)
127.0.0.1:6379> SET device:AMB-001:presence ONLINE EX 60
OK
```

Clave → Valor

Clave

device:AMB-001:state



Redis

Almacenamiento
en memoria

Valor

ONLINE

- ✓ Operaciones simples y rápidas
- ✓ Ideal para datos temporales
- ✓ Soporta expiración (TTL)



Redis es un ejemplo de tecnología que permite trabajar eficientemente con estructuras en memoria y datos temporales.

Fortalezas y límites

Excelente para algunos accesos, incómodo para otros



Puede resultar útil para:

- último estado;
- sesiones;
- caché;
- locks o coordinaciones simples;
- datos temporales;
- valores de acceso directo.

Acceso directo rápido

Con una clave conocida, obtenemos el valor al instante.

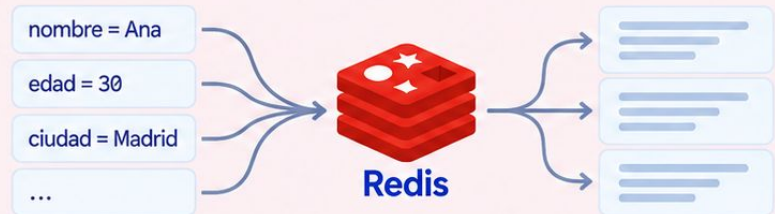


Puede resultar poco conveniente cuando necesitamos:

- relaciones complejas;
- búsquedas arbitrarias;
- consultas por muchos atributos.

Búsqueda compleja

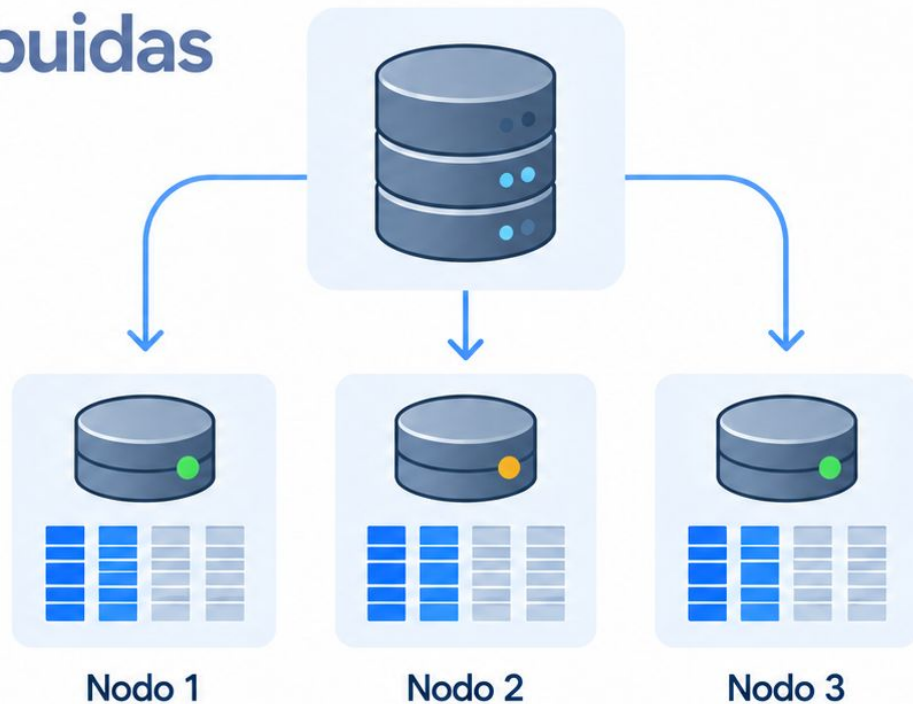
Requiere evaluar múltiples atributos y no es un acceso directo.



Clave-valor funciona mejor cuando el patrón principal de acceso comienza con una clave conocida.

5

Bases Columnares Distribuidas



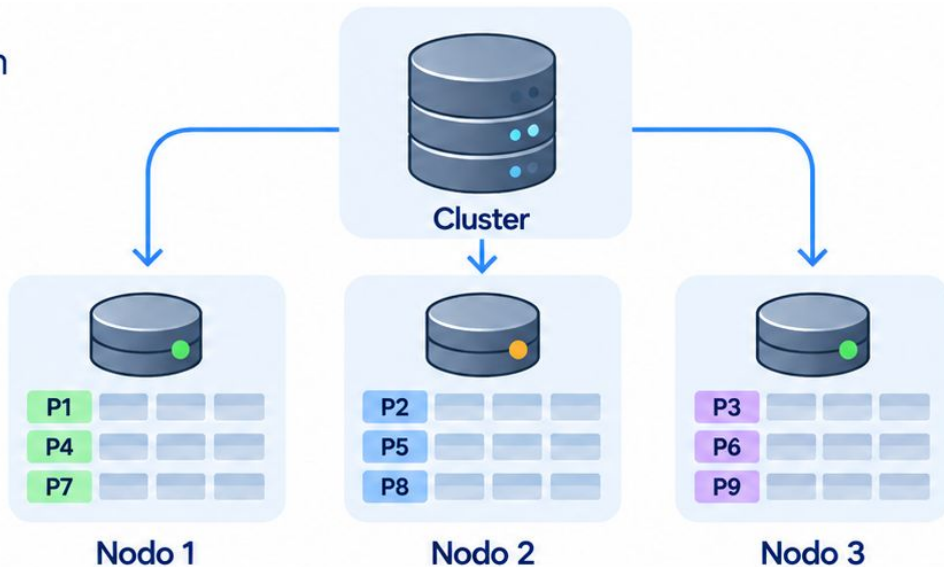
Wide-column

Diseñadas pensando en distribución y patrones de acceso

Tecnologías como Cassandra o ScyllaDB organizan los datos para permitir:

- grandes volúmenes;
- muchas operaciones;
- distribución entre nodos;
- particionamiento;
- escalabilidad horizontal.

El modelo de consulta influye fuertemente en la estructura.



En este modelo solemos diseñar las estructuras a partir de cómo necesitamos consultar los datos.

No confundir dos conceptos

Aquí hablamos de bases wide-column distribuidas

En esta clase

Wide-column / column-family

Bases distribuidas para grandes volúmenes

Ejemplos:

- Cassandra
- ScyllaDB



Datos particionados y distribuidos entre nodos para escalar horizontalmente.

≠

No estamos hablando de

Almacenamiento columnar analítico

Motores que almacenan físicamente datos por columnas

Ejemplos:

- ClickHouse
- Amazon Redshift
- Google BigQuery
- Snowflake

id	fecha	temperatura	humedad

Optimizados para análisis y procesamiento de grandes volúmenes de datos.



El nombre puede resultar confuso: el foco aquí está en el modelo distribuido wide-column.

¿Dónde queda cada dato?

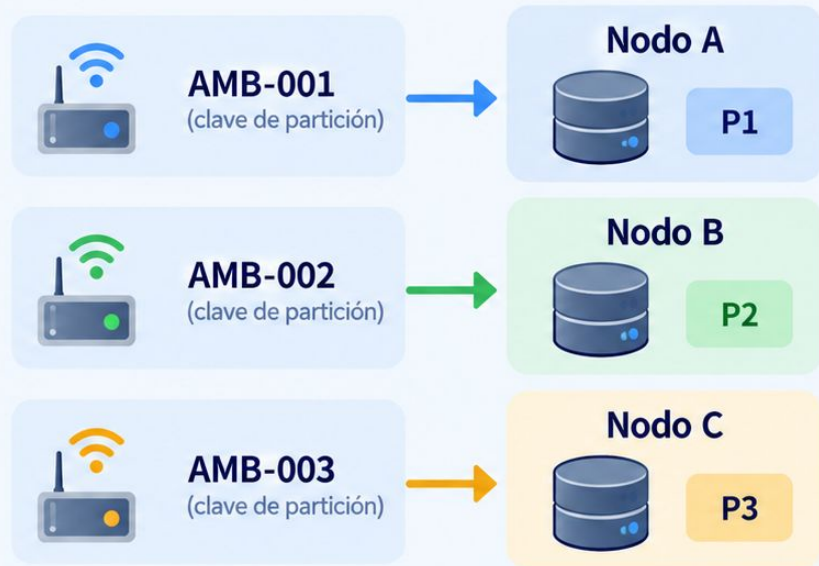
La clave de partición distribuye la información

Ejemplo conceptual:

`device_id = AMB-001`

puede determinar en qué partición se almacenan sus mediciones.

Otros dispositivos pueden quedar en otras particiones.



La clave de partición afecta directamente cómo se distribuyen y recuperan los datos.

Diseñar desde el patrón de acceso

¿Qué pregunta debemos responder eficientemente?

Supongamos que necesitamos frecuentemente:

“Obtener las mediciones de un dispositivo durante un período, ordenadas por tiempo.”

Podemos diseñar una estructura específicamente orientada a:

`device_id + timestamp`

Consulta



mediciones de
AMB-001 entre
10:00 y 11:00

Clave de
partición



`device_id =
AMB-001`

Registros ordenados
por tiempo

Timestamp	Temperatura
10:00	24,2 °C
10:05	24,3 °C
10:10	24,5 °C
10:15	24,4 °C



En bases distribuidas de este tipo, conocer las consultas antes de diseñar resulta fundamental.

Mediciones por dispositivo

Ejemplo IoT: Un patrón de acceso conocido

device_id: AMB-001

10:30 → temperatura → 24.2

10:31 → temperatura → 24.4

10:32 → temperatura → 24.6

La estructura puede quedar orientada directamente a recuperar la secuencia correspondiente a ese dispositivo.

Partición de datos

device_id = AMB-001

Registros
ordenados
por tiempo

Timestamp	Variable	Valor
10:30	temperatura	24.2
10:31	temperatura	24.4
10:32	temperatura	24.6



Los datos se organizan para que las consultas principales puedan resolverse de manera eficiente y distribuida.

Desnormalizar para consultar

La misma información puede aparecer en más de una estructura

Podríamos mantener:

Estructura A

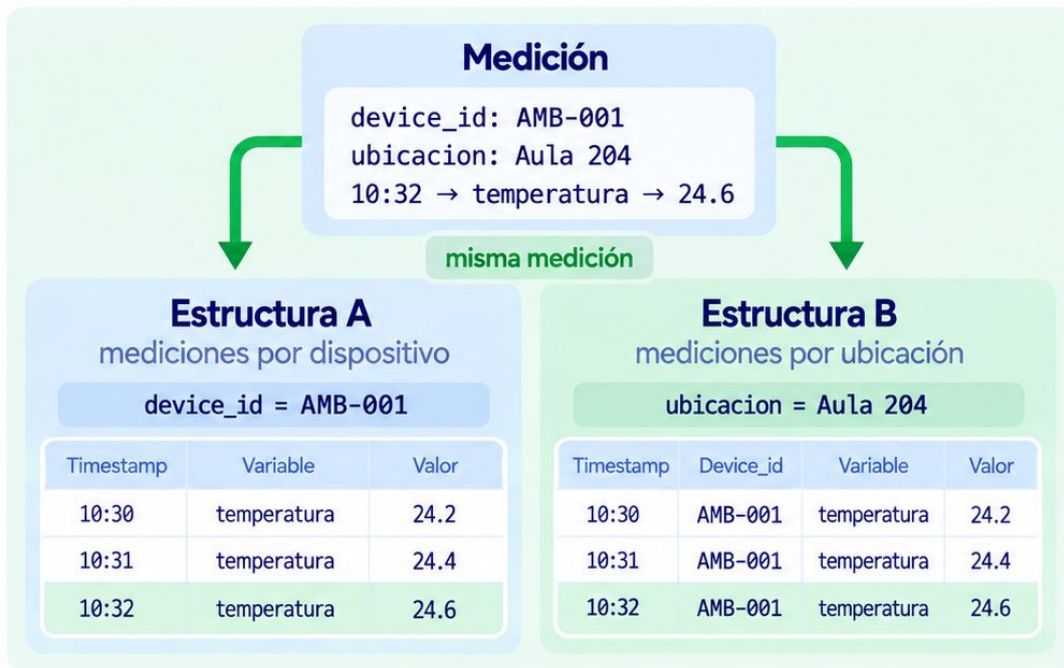
mediciones por dispositivo.

Estructura B

mediciones por ubicación.

Eso puede implicar duplicación.

Pero cada estructura responde eficientemente a una consulta determinada.



En algunos modelos NoSQL, duplicar datos puede ser una decisión deliberada de diseño.

Muchas particiones, muchos nodos

El diseño debe evitar concentrar todo en el mismo lugar

Una mala clave de partición puede provocar:

- particiones demasiado grandes;
- carga desigual;
- nodos muy utilizados;
- puntos calientes.

Una buena distribución busca repartir datos y operaciones.

Distribución desbalanceada

Un nodo concentra la mayor parte de la carga



Distribución equilibrada

Los datos y operaciones se reparten entre nodos



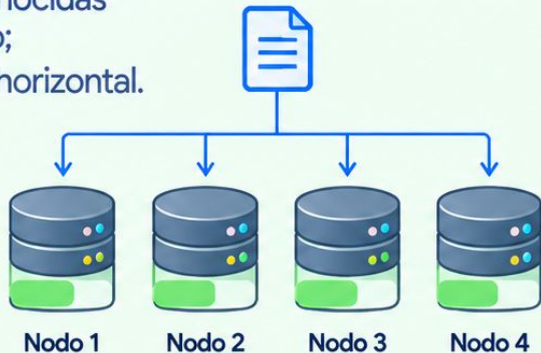
Escalar horizontalmente requiere distribuir correctamente la carga, no solamente agregar servidores.

Fortalezas y límites

Distribución a gran escala con patrones conocidos

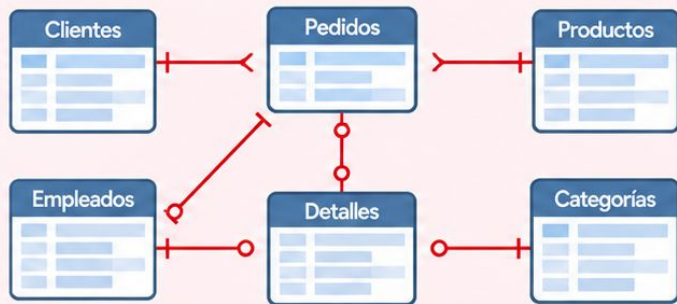
Puede resultar interesante cuando existen:

- grandes volúmenes;
- alta tasa de escritura;
- necesidad de distribución;
- consultas conocidas de antemano;
- crecimiento horizontal.



Puede resultar menos cómodo para:

- consultas arbitrarias;
- relaciones complejas;
- JOIN tradicionales;
- exploración flexible del modelo.



La escalabilidad se obtiene aceptando determinadas restricciones en la forma de modelar y consultar.

6

Bases de Grafos



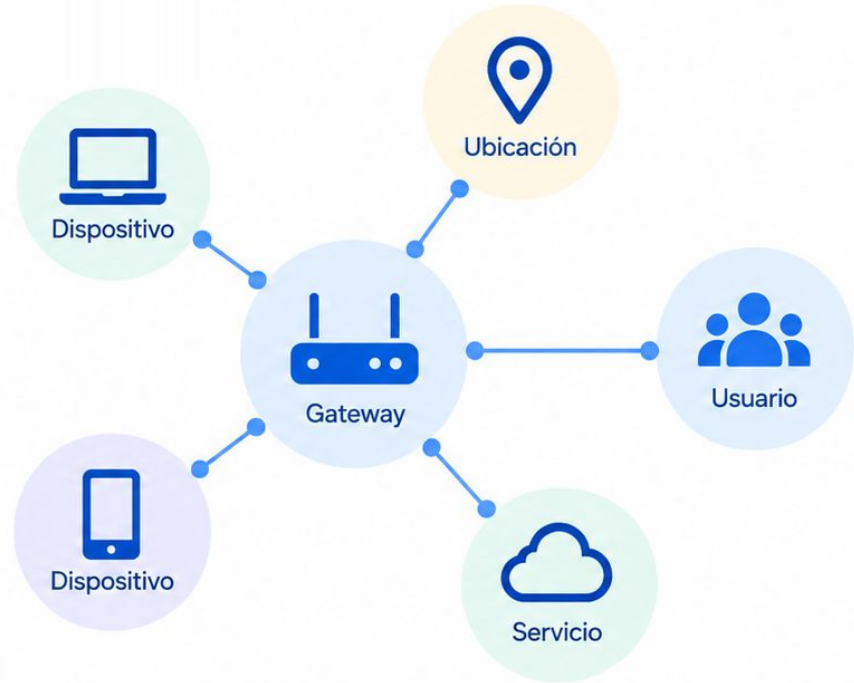
Modelo de grafos

Representar elementos y conexiones

Un grafo puede representar:

- dispositivos;
- gateways;
- ubicaciones;
- usuarios;
- servicios;
- dependencias;
- conexiones.

Pero el foco no está solamente en los elementos.
Está especialmente en **cómo se relacionan**.



En un grafo, las relaciones forman parte central del modelo.

Tres elementos básicos

Una representación sencilla

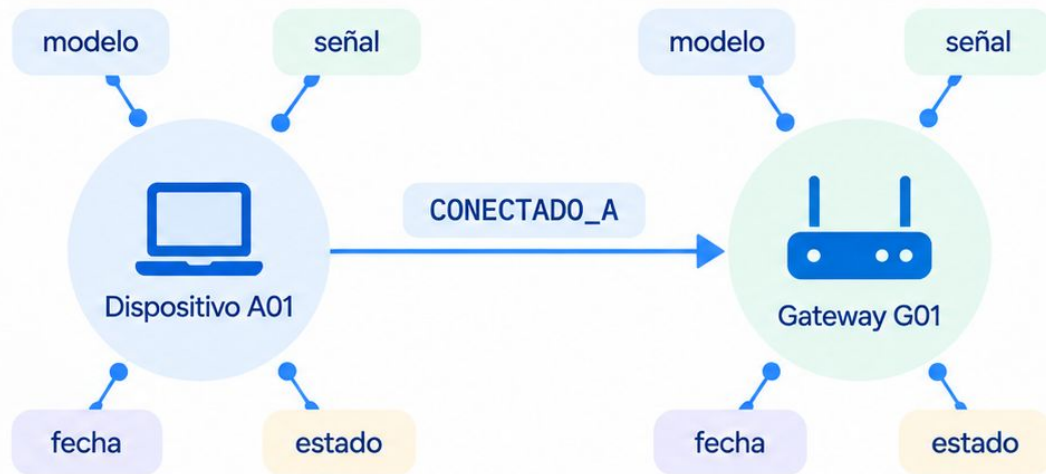
Nodo: Dispositivo A01

Relación: CONECTADO_A

Nodo: Gateway G01

Propiedades:

- modelo;
- señal;
- fecha;
- estado.



Los nodos representan elementos y las relaciones representan cómo se vinculan.

Representar una red

Dispositivos, gateways y ubicaciones

Ubicación A

↓ CONTIENE

Gateway G01

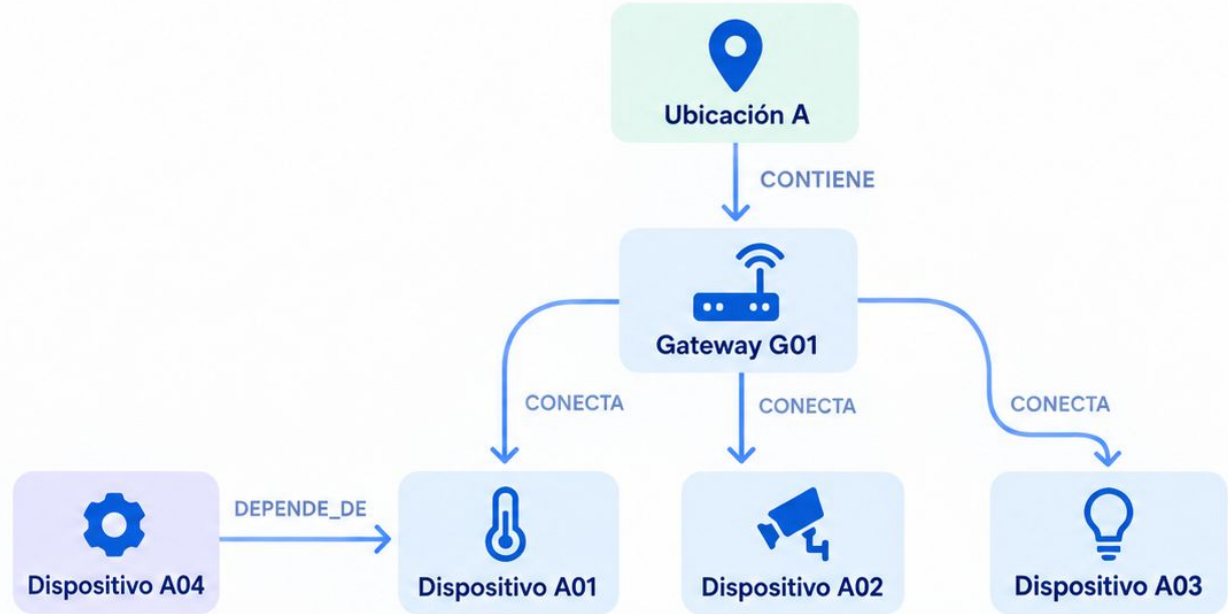
↓ CONECTA

Dispositivo A01

Dispositivo A02

Dispositivo A03

Otro dispositivo puede depender de A01 o comunicarse mediante otro camino.



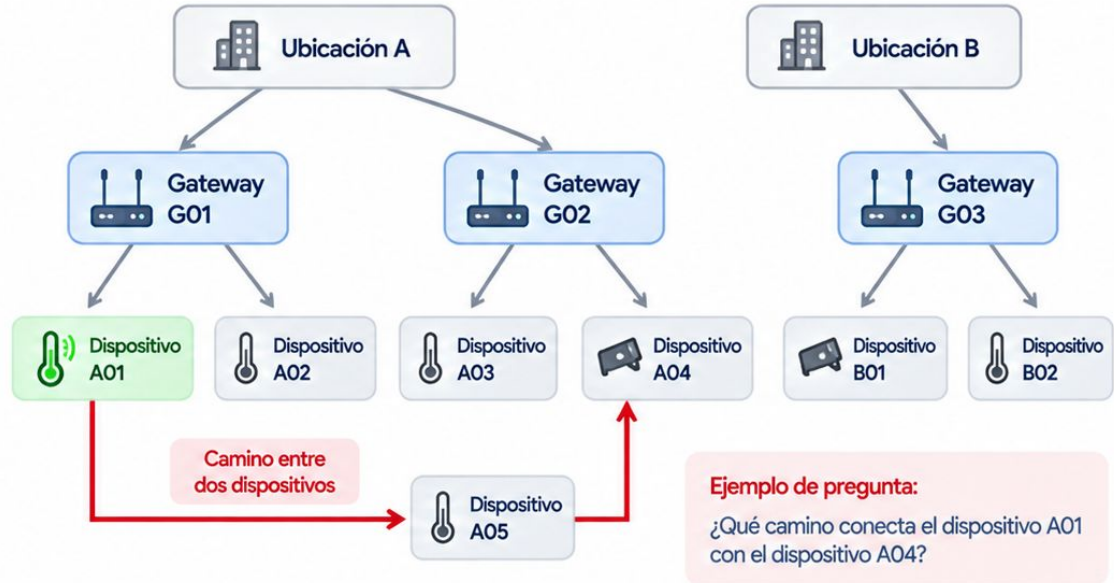
Un grafo permite representar de manera natural estructuras donde las conexiones también tienen significado.

Preguntas basadas en caminos

No siempre buscamos solamente registros

Podemos querer saber:

- ¿qué dispositivos dependen de este gateway?;
- ¿qué componentes quedarían afectados si falla?;
- ¿qué camino conecta dos dispositivos?;
- ¿qué elementos se encuentran a dos relaciones de distancia?;
- ¿qué dispositivos comparten infraestructura?



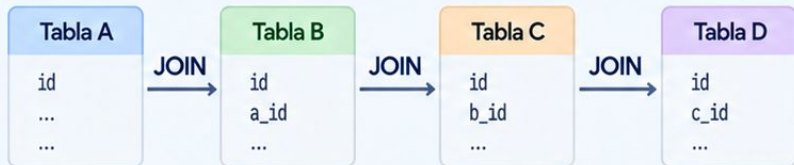
Cuando la pregunta implica recorrer relaciones, un grafo puede representar el problema de manera muy directa.

Relaciones profundas

El modelo cambia la forma de pensar la consulta

Modelo relacional

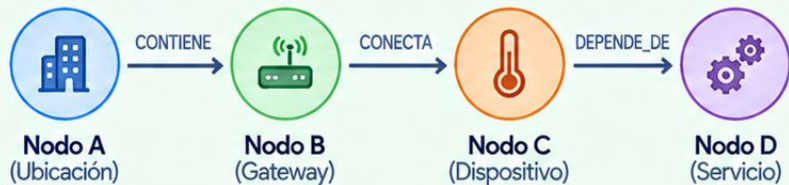
Varios JOIN para llegar a la información



```
SELECT a.*, b.*, c.*, d.*  
FROM   tabla_a a  
JOIN   tabla_b b ON b.a_id = a.id  
JOIN   tabla_c c ON c.b_id = b.id  
JOIN   tabla_d d ON d.c_id = c.id  
WHERE ...;
```

Modelo de grafos

Recorrer relaciones entre nodos



```
MATCH (a:Ubicacion)-[:CONTIENE]->(b:Gateway)  
      -[:CONECTA]->(c:Dispositivo)  
      -[:DEPENDE_DE]->(d:Servicio)  
RETURN a, b, c, d;
```



Los dos modelos pueden representar relaciones.

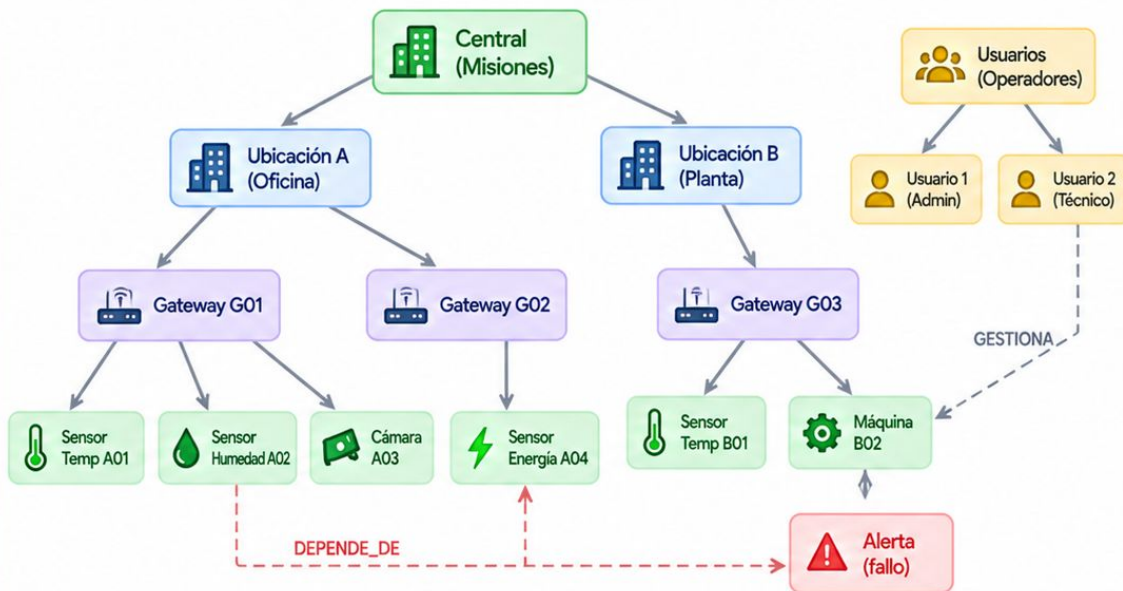
La diferencia aparece especialmente cuando las conexiones y recorridos dominan las consultas.

Casos para grafos

Cuando necesitamos comprender conexiones

Por ejemplo:

- topologías de red;
- dependencias entre componentes;
- rutas;
- relaciones entre usuarios;
- jerarquías complejas;
- impacto de fallos;
- detección de comunidades o conexiones.



Un grafo es interesante cuando las relaciones son tan importantes como los propios datos.

Más allá de los grafos

Una FK no justifica cambiar de tecnología

Si solamente
necesitamos:

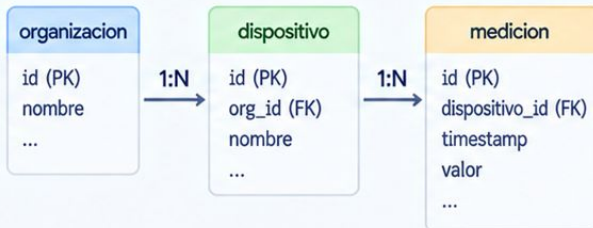
- Organización 1:N
Dispositivo;
- Dispositivo 1:N
Medición;
- algunos JOIN conocidos;

una base relacional
puede representar
el problema
perfectamente.



Modelo relacional

Relaciones simples y bien definidas



Consultas típicas con JOIN conocidos
y estructura estable.



Modelo de grafos

Más útil cuando las relaciones son complejas



Útil cuando hay muchas conexiones,
recorridos y dependencias complejas.



Tener relaciones no significa automáticamente necesitar una base de grafos.

7

Elegir y Combinar Modelos



Un sistema, varias formas

El modelo depende de qué necesitamos hacer



No estamos representando cuatro sistemas diferentes.

Estamos observando cuatro necesidades diferentes del mismo sistema.

¿Cómo son los datos?

La forma de la información aporta una primera pista

Preguntas:

- ¿la estructura es estable?;
- ¿varía entre elementos?;
- ¿es jerárquica?;
- ¿las relaciones dominan el problema?;
- ¿accedemos mediante una clave?;
- ¿generamos enormes secuencias de registros?

Tabla



Documento



Clave-valor



clave → valor

Partición



Grafo



La estructura del dato ayuda a descartar o considerar modelos, pero no alcanza por sí sola.

¿Cómo vamos a consultar?

Los patrones de acceso condicionan el diseño

Debemos saber si necesitamos:

- búsquedas por identificador;
- consultas flexibles;
- relaciones;
- recorridos;
- rangos;
- agregaciones;
- lectura del documento completo;
- grandes tasas de escritura.



Elegir almacenamiento sin conocer las consultas futuras suele producir malos diseños.

Foco en la necesidad

Consistencia, disponibilidad y distribución

También debemos analizar:

- ¿puede existir temporalmente información desactualizada?
- ¿la operación debe confirmarse en todos los nodos?
- ¿qué ocurre si un nodo falla?
- ¿qué ocurre si se corta la comunicación?
- ¿qué latencia necesitamos?
- ¿cuánto debe escalar?



Consistencia

¿Qué tan actualizada debe estar la información?

- Consistencia fuerte
- Consistencia eventual
- Lecturas replicadas



Disponibilidad

¿El sistema debe estar siempre disponible?

- Tolerancia a fallos
- Alta disponibilidad
- Degradación controlada



Latencia

¿Qué tiempo de respuesta necesitamos?

- Tiempo real
- Baja latencia
- Procesamiento asíncrono



Distribución y escala

¿Cómo crece el sistema?

- Escalado horizontal
- Particionado de datos
- Replicación geográfica



La tecnología debe responder a las garantías que necesita el negocio.

No es una competencia

Son modelos con diferentes fortalezas



Relacional (SQL)

Muy útil para:



Integridad

Datos consistentes y válidos



Relaciones

Datos relacionados de forma estructurada



Transacciones

Operaciones seguras y confiables



Consultas flexibles

SQL potente y expresivo



Estructura estable

Esquemas definidos y bien controlados



Diferentes fortalezas
para diferentes necesidades



NoSQL

Puede aportar ventajas para determinados casos de:



Flexibilidad

Esquemas dinámicos y cambiantes



Acceso directo

Lecturas y escrituras simples y rápidas



Distribución

Diseñados para entornos distribuidos



Relaciones complejas

Conexiones y recorridos (por ejemplo, grafos)



Escalabilidad específica

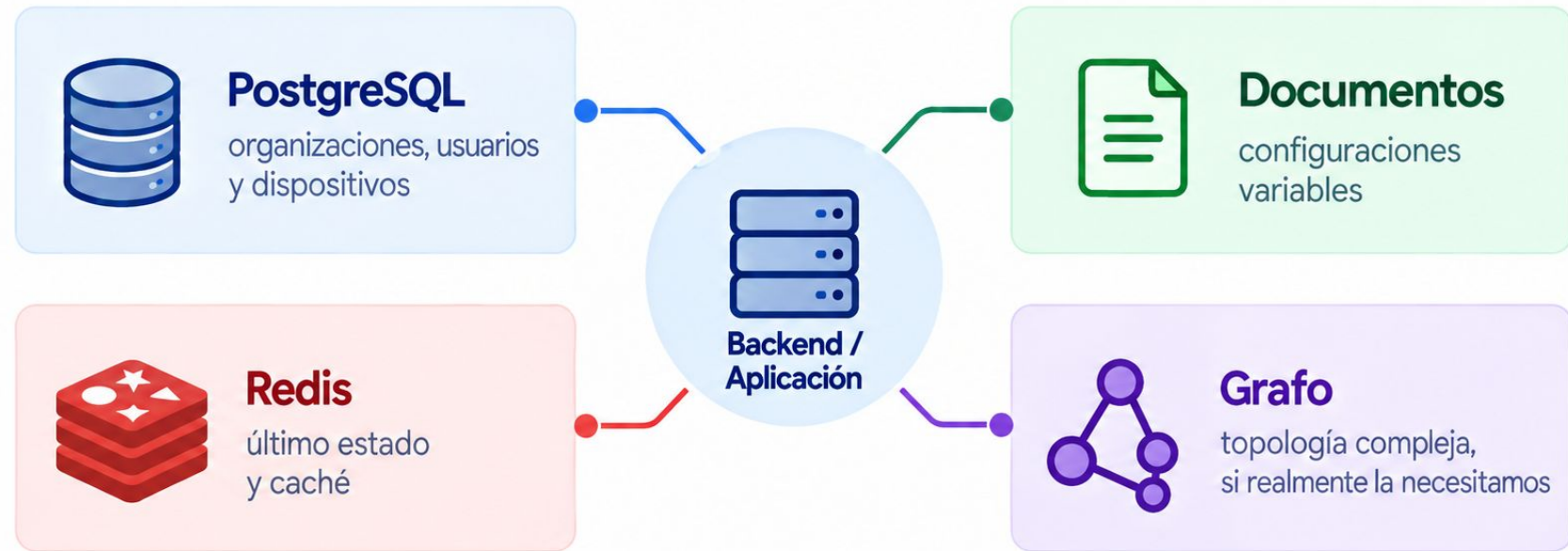
Altos volúmenes de datos o escrituras



La decisión no es SQL o NoSQL por preferencia. Depende del problema.

Solución con varias tecnologías

Cada mecanismo para una necesidad concreta



Persistencia políglota significa combinar mecanismos cuando existen requerimientos diferentes que lo justifican.

Una posible evolución

Agregar solamente lo necesario



Una arquitectura puede evolucionar progresivamente sin incorporar toda la complejidad desde el comienzo.

Sobrediseño

Cada componente también tiene un costo



Agregar una base adicional implica:

- desplegarla;
- configurarla;
- protegerla;
- monitorearla;
- respaldarla;
- actualizarla;
- integrar sus datos;
- aprender a operarla.

1

Arquitectura simple

Soluciona la necesidad con lo necesario.



2

Arquitectura sobrediseñada

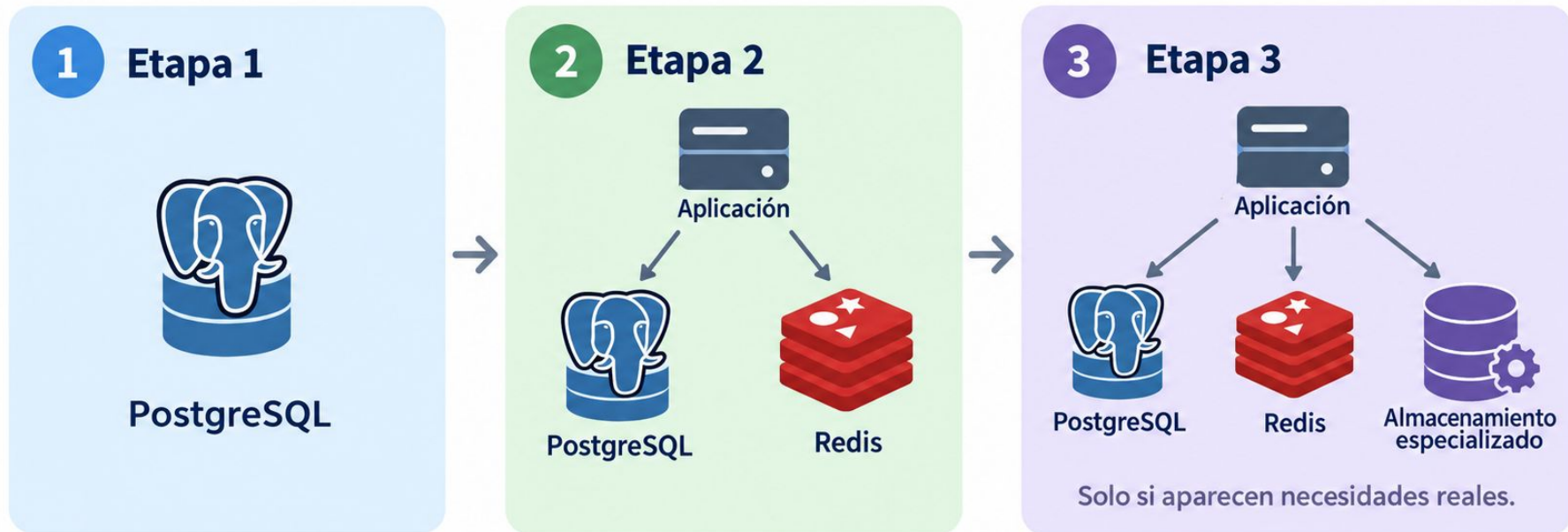
Más componentes, más complejidad, más costos, sin un beneficio claro.



Una nueva tecnología debería resolver un problema suficientemente importante como para justificar su complejidad.

Empezar simple

Incorporar complejidad cuando aparezca el requerimiento



La arquitectura debe crecer junto con los requerimientos, no adelantarse innecesariamente a ellos.

8

Práctica Guiada

Modelar el mismo sistema mediante diferentes paradigmas.



Práctica

El mismo problema, diferentes representaciones

Vamos a analizar un sistema con:

- organizaciones;
- ubicaciones;
- dispositivos;
- configuraciones variables;
- estado actual;
- mediciones;
- gateways;
- relaciones entre componentes.

Para cada necesidad debemos decidir:

¿qué modelo resulta razonable y por qué?



La práctica no busca encontrar una única respuesta correcta, sino justificar técnicamente cada decisión.

Actividad

Representar cada necesidad

En esta actividad diseñaremos diferentes representaciones de un mismo sistema, analizando cómo modelar cada necesidad de forma adecuada.

Cada parte aborda un tipo de representación y un patrón de acceso diferente.

Piensa en el modelo de datos más adecuado y justifica tu propuesta.



Parte A — Documental

Representar la configuración de distintos modelos de dispositivos mediante documentos.



Parte B — Clave-valor

Representar:

- último estado
 - última medición
 - última conexión
- utilizando claves.



Parte C — Grafo

Representar:
Ubicación → Gateway → Dispositivos
junto con dependencias entre componentes.



Parte D — Wide-column

Proponer cómo organizaríamos millones de mediciones si la consulta principal fuera:
“mediciones de un dispositivo durante un período”.



Cada representación debe responder a un patrón de acceso concreto.

Comparar antes de elegir

Ventajas, límites y justificación

Para cada alternativa responder:

- 1 ¿Qué problema resuelve bien?
- 2 ¿Cómo se identifica la información?
- 3 ¿Cómo se consulta?
- 4 ¿Qué información duplicamos?
- 5 ¿Qué reglas debemos mantener?
- 6 ¿Qué ocurre si cambian los requerimientos?
- 7 ¿Realmente necesitamos una tecnología adicional?

Criterio	 Documental	 Clave-valor	 Grafo	 Wide-column
Problema	✓	✓	✓	✓
Identificación	✓	✓	—	✓
Consulta	—	✓	✓	✓
Duplicación	—	✓	✗	—
Reglas	✓	—	✓	—
Cambios	✓	✓	—	✓
Tecnología adicional	—	✗	✓	—



Seleccionar una tecnología también significa explicar por qué sus costos y limitaciones son aceptables para ese escenario.



Para ir cerrando..

Errores frecuentes

Problemas típicos al trabajar con NoSQL

- 1 Elegir NoSQL solamente porque los datos están en JSON.
- 2 Suponer que NoSQL significa trabajar sin esquema ni reglas.
- 3 Elegir una tecnología antes de conocer las consultas necesarias.
- 4 Embebir información que puede crecer indefinidamente.
- 5 Duplicar datos sin considerar cómo mantenerlos consistentes.
- 6 Utilizar almacenamiento temporal sin analizar qué ocurre si los datos se pierden.
- 7 Elegir una mala clave de partición y concentrar la carga.
- 8 Utilizar una base de grafos solamente porque existen relaciones.
- 9 Incorporar varias bases de datos sin una necesidad concreta.



No es solo JSON



Sin esquema ni reglas



Sin conocer
las consultas



Datos que
crecen indefinidamente



Mala clave de partición
y carga concentrada



Base de grafos
sin necesidad real



Duplicar datos sin
mantener consistencia



Varias bases de datos
sin una necesidad concreta



Utilizar otro modelo de almacenamiento debe resolver un problema real, no solamente cambiar la tecnología.

Buenas prácticas

Diseñar NoSQL con criterio

- 1 Comenzar por los requerimientos.
- 2 Conocer los patrones de acceso.
- 3 Diseñar documentos alrededor de unidades de información coherentes.
- 4 Validar también las estructuras flexibles.
- 5 Diseñar cuidadosamente las claves.
- 6 Definir qué información puede expirar.
- 7 Elegir claves de partición que distribuyan correctamente los datos.
- 8 Analizar qué garantías de consistencia necesita cada operación.
- 9 Incorporar una nueva tecnología solamente cuando aporte un beneficio concreto.



NoSQL cambia algunas decisiones de modelado, pero no elimina la necesidad de diseñar.

Antes de elegir una tecnología

Preguntas para tomar una decisión

- 1 ¿Qué información necesitamos almacenar?
- 2 ¿La estructura es estable o variable?
- 3 ¿Cómo vamos a consultar los datos?
- 4 ¿Necesitamos relaciones o recorridos complejos?
- 5 ¿Accedemos principalmente mediante una clave?
- 6 ¿Qué volumen y frecuencia de escritura esperamos?
- 7 ¿Necesitamos distribuir los datos?
- 8 ¿Qué nivel de consistencia necesitamos?
- 9 ¿Los datos pueden expirar?
- 10 ¿La solución actual ya puede resolver razonablemente el problema?



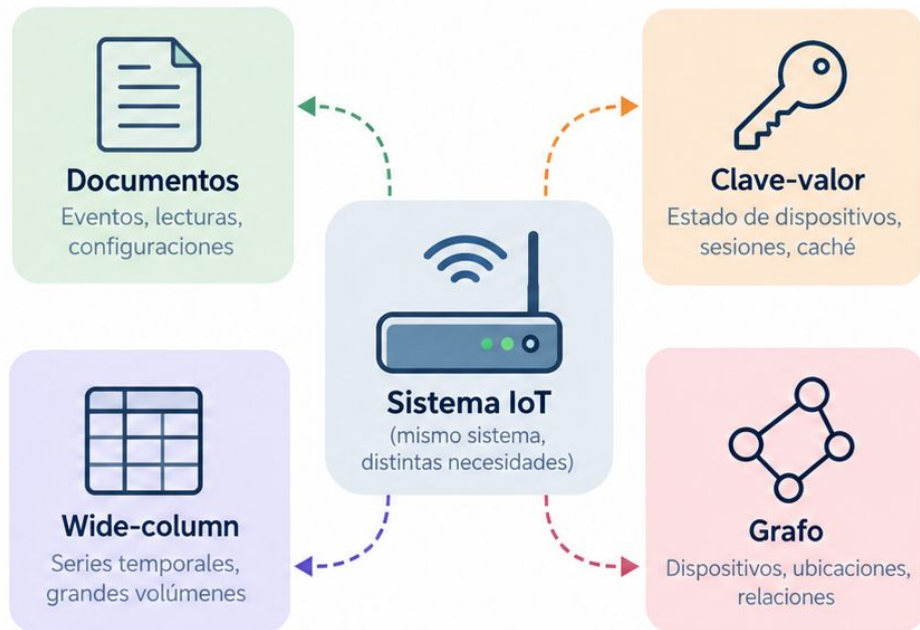
La tecnología debería aparecer después de comprender los datos, las consultas y los requerimientos del sistema.

¿Qué observamos?

Un mismo sistema puede necesitar diferentes modelos

Durante la clase:

- comprendimos por qué aparecieron las bases NoSQL;
- analizamos la flexibilidad de esquema;
- relacionamos distribución, disponibilidad y consistencia;
- introdujimos conceptualmente CAP;
- modelamos información mediante documentos;
- analizamos el modelo clave-valor;
- estudiamos particionamiento en bases wide-column;
- representamos relaciones mediante grafos;
- diferenciamos JSONB de una base documental;
- analizamos cuándo combinar diferentes mecanismos de persistencia.

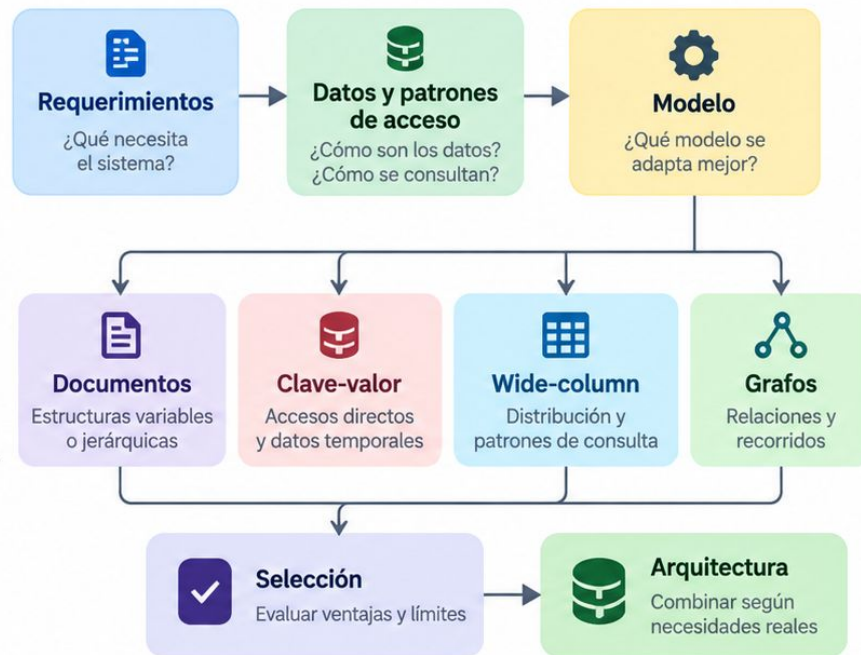


Cambiar de modelo de almacenamiento también cambia la manera en que organizamos y consultamos los datos.

Resumen de la clase

Ideas principales

- NoSQL agrupa diferentes modelos de almacenamiento.
- Flexibilidad de esquema no significa ausencia de estructura.
- Distribuir datos introduce decisiones sobre disponibilidad y consistencia.
- CAP ayuda a comprender qué ocurre ante particiones de red.
- Las bases documentales resultan útiles para estructuras variables o jerárquicas.
- Clave-valor facilita accesos directos y almacenamiento temporal.
- Las bases wide-column priorizan distribución y patrones de consulta conocidos.
- Los grafos ponen las relaciones y recorridos en el centro del modelo.
- JSONB y una base documental no representan la misma decisión arquitectónica.
- SQL y NoSQL pueden coexistir.
- La persistencia políglota debe responder a necesidades reales.



La elección de una base de datos depende de cómo son los datos, cómo necesitamos utilizarlos y qué requerimientos debe cumplir el sistema.

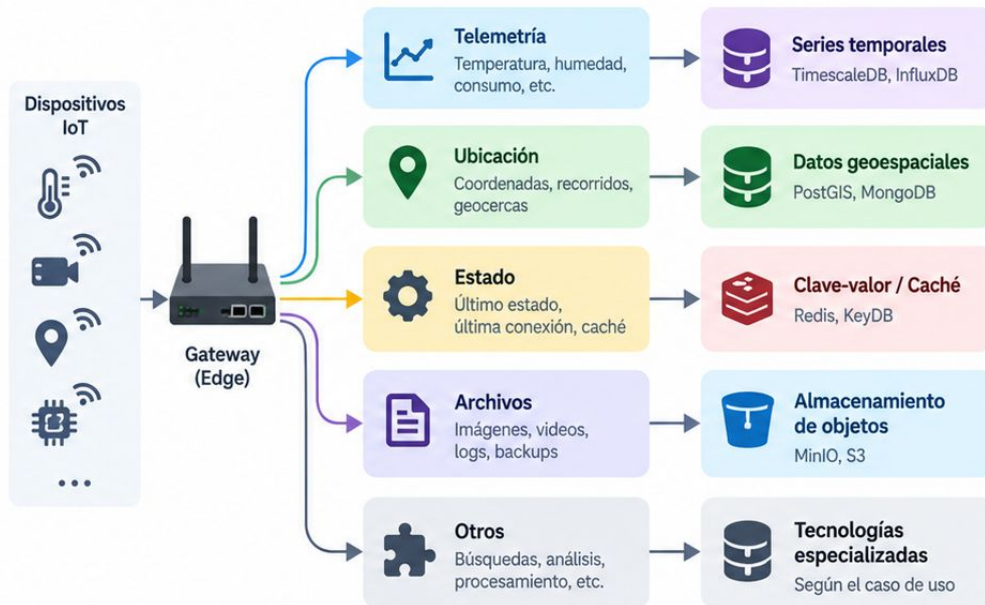
Próxima clase

Bases de Datos y Almacenamiento Especializado para IoT

En esta clase vimos diferentes modelos NoSQL.

En la próxima vamos a concentrarnos en problemas especialmente frecuentes en IoT:

- grandes secuencias de mediciones;
- series temporales;
- retención y reducción de resolución;
- agregaciones temporales;
- datos geoespaciales;
- consultas de distancia y proximidad;
- caché y almacenamiento temporal;
- archivos y almacenamiento de objetos;
- otras tecnologías especializadas.



Hoy ampliamos los modelos disponibles.

La próxima clase veremos mecanismos de almacenamiento especializados para determinados tipos de datos y consultas IoT.



Enlace a la encuesta



Muchas gracias ¿Dudas?